

OOP v2.0 - Studentu Sistema

2.0

Generated by Doxygen 1.17.0

1 Directory Hierarchy	1
1.1 Directories	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 File Index	7
4.1 File List	7
5 Directory Documentation	9
5.1 include Directory Reference	9
5.2 src Directory Reference	9
6 Class Documentation	11
6.1 Studentas Class Reference	11
6.1.1 Detailed Description	13
6.1.2 Constructor & Destructor Documentation	13
6.1.2.1 Studentas() [1/3]	13
6.1.2.2 ~Studentas()	13
6.1.2.3 Studentas() [2/3]	13
6.1.2.4 Studentas() [3/3]	13
6.1.3 Member Function Documentation	13
6.1.3.1 getEgz()	13
6.1.3.2 getGalutinisMed()	14
6.1.3.3 getGalutinisVid()	14
6.1.3.4 getPavarde()	14
6.1.3.5 getPaz()	14
6.1.3.6 getRez()	14
6.1.3.7 getVardas()	14
6.1.3.8 isvalytiPaz()	14
6.1.3.9 operator=() [1/2]	15
6.1.3.10 operator=() [2/2]	15
6.1.3.11 pridetiPaz()	15
6.1.3.12 print()	15
6.1.3.13 read()	15
6.1.3.14 setEgz()	15
6.1.3.15 setGalutinisMed()	16
6.1.3.16 setGalutinisVid()	16
6.1.3.17 setPavarde()	16
6.1.3.18 setPaz()	16
6.1.3.19 setRez()	16

6.1.3.20 setVardas()	16
6.1.4 Member Data Documentation	17
6.1.4.1 egz	17
6.1.4.2 galutinisMed	17
6.1.4.3 galutinisVid	17
6.1.4.4 paz	17
6.1.4.5 rez	17
6.2 Zmogus Class Reference	17
6.2.1 Detailed Description	18
6.2.2 Constructor & Destructor Documentation	18
6.2.2.1 Zmogus() [1/3]	18
6.2.2.2 Zmogus() [2/3]	19
6.2.2.3 Zmogus() [3/3]	19
6.2.2.4 ~Zmogus()	19
6.2.3 Member Function Documentation	19
6.2.3.1 getPavarde()	19
6.2.3.2 getVardas()	19
6.2.3.3 print()	19
6.2.3.4 read()	20
6.2.3.5 setPavarde()	20
6.2.3.6 setVardas()	20
6.2.4 Friends And Related Symbol Documentation	20
6.2.4.1 operator<<	20
6.2.4.2 operator>>	20
6.2.5 Member Data Documentation	20
6.2.5.1 pavarde	20
6.2.5.2 vardas	20
7 File Documentation	21
7.1 include/funkcijos.h File Reference	21
7.1.1 Function Documentation	22
7.1.1.1 generuotiFaila()	22
7.1.1.2 inputas()	22
7.1.1.3 laikoSkaiciavimas()	22
7.1.1.4 mediana()	22
7.1.1.5 nuskaitymas()	22
7.1.1.6 outputas()	22
7.1.1.7 padalintiStudentus1()	23
7.1.1.8 padalintiStudentus2()	23
7.1.1.9 padalintiStudentus3()	23
7.1.1.10 rusiavimas()	23
7.1.1.11 spausdinimas()	23

7.1.1.12 spausdintiFaila()	23
7.1.1.13 tyrimas1()	24
7.1.1.14 tyrimas2()	24
7.1.1.15 vidurkis()	24
7.2 funkcijos.h	24
7.3 include/Studentas.h File Reference	28
7.3.1 Detailed Description	28
7.4 Studentas.h	29
7.5 include/Zmogus.h File Reference	30
7.5.1 Detailed Description	31
7.6 Zmogus.h	31
7.7 src/funkcijos.cpp File Reference	32
7.7.1 Function Documentation	32
7.7.1.1 generuotiFaila()	32
7.7.1.2 inputas()	32
7.7.1.3 mediana()	32
7.7.1.4 outputas()	32
7.7.1.5 spausdinimas()	33
7.7.1.6 tyrimas1()	33
7.7.1.7 tyrimas2()	33
7.7.1.8 vidurkis()	33
7.8 src/main.cpp File Reference	33
7.8.1 Function Documentation	34
7.8.1.1 main()	34
7.8.2 Variable Documentation	34
7.8.2.1 pavardes	34
7.8.2.2 vardai	34
Index	35

Chapter 1

Directory Hierarchy

1.1 Directories

include	9
funkcijos.h	21
Studentas.h	28
Zmogus.h	30
src	9
funkcijos.cpp	32
main.cpp	33

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Zmogus	17
Studentas	11

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Studentas

Studento klase, paveldinti is [Zmogus](#). Saugo varda, pavarde, pazymius, egzamino rezultata ir galutinius ivertinimus 11

Zmogus

Abstrakti bazine klase, aprasanti zmogu. Negalima tiesiogiai sukurti jos objektu 17

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

include/funkcijos.h	21
include/Studentas.h	
Studento klases aprasas	28
include/Zmogus.h	
Abstrakti bazine klase zmogui	30
src/funkcijos.cpp	32
src/main.cpp	33

Chapter 5

Directory Documentation

5.1 include Directory Reference

Files

- file [funkcijos.h](#)
- file [Studentas.h](#)
Studento klases aprasas.
- file [Zmogus.h](#)
Abstrakti bazine klase zmogui.

5.2 src Directory Reference

Files

- file [funkcijos.cpp](#)
- file [main.cpp](#)

Chapter 6

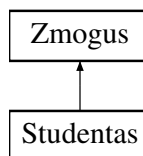
Class Documentation

6.1 Studentas Class Reference

Studento klase, paveldinti is [Zmogus](#). Saugo varda, pavarde, pazymius, egzamino rezultata ir galutinius ivertinimus.

```
#include <Studentas.h>
```

Inheritance diagram for Studentas:



Public Member Functions

- [Studentas](#) ()
Numatytasis konstruktorius.
- [~Studentas](#) () override
Destruktorius.
- [Studentas](#) (const [Studentas](#) &kitas)
Kopijavimo konstruktorius.
- [Studentas](#) ([Studentas](#) &&kitas)
Perkelimo konstruktorius.
- [Studentas](#) & [operator=](#) (const [Studentas](#) &kitas)
Kopijavimo priskyrimo operatorius.
- [Studentas](#) & [operator=](#) ([Studentas](#) &&kitas)
Perkelimo priskyrimo operatorius.
- const std::string & [getVardas](#) () const override
Grazina varda.
- const std::string & [getPavarde](#) () const override
Grazina pavarde.
- void [setVardas](#) (const std::string &v) override
Nustato varda.
- void [setPavarde](#) (const std::string &p) override

- *Nustato pavarde.*
- void `print` (std::ostream &os) const override
Spausdina studento duomenis i srauta.
- void `read` (std::istream &in) override
Nuskaito studento duomenis is srauto.
- const std::vector< int > & `getPaz` () const
Grazina pazymiu vektoriui.
- int `getEgz` () const
Grazina egzamino pazymi.
- double `getRez` () const
Grazina rezultata.
- double `getGalutinisVid` () const
Grazina galutini vidurkiu.
- double `getGalutinisMed` () const
Grazina galutini mediana.
- void `setPaz` (const std::vector< int > &naujiPaz)
Nustato pazymius.
- void `setEgz` (int naujasEgz)
Nustato egzamino pazymi.
- void `setRez` (double naujasRez)
Nustato rezultata.
- void `setGalutinisVid` (double naujasGalutinisVid)
Nustato galutini vidurkiu.
- void `setGalutinisMed` (double naujasGalutinisMed)
Nustato galutini mediana.
- void `pridetiPaz` (int pazymys)
Prideda viena pazymi.
- void `isvalytiPaz` ()
Isvalo pazymius.

Public Member Functions inherited from `Zmogus`

- `Zmogus` ()=default
Numatytasis konstruktorius.
- `Zmogus` (const std::string &vardas, const std::string &pavarde)
Konstruktorius su parametrais.
- `Zmogus` (`Zmogus` &&kitas)
Perkelimo konstruktorius.
- virtual `~Zmogus` ()=default
Virtualus destruktorius.

Private Attributes

- std::vector< int > `paz`
Namu darbu pazymiai.
- int `egz`
Egzamino pazymys.
- double `rez`
Rezultatas.
- double `galutinisVid`
Galutinis pagal vidurkis.
- double `galutinisMed`
Galutinis pagal mediana.

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- std::string [vardas](#)
Zmogaus vardas.
- std::string [pavarde](#)
Zmogaus pavarde.

6.1.1 Detailed Description

Studento klase, paveldinti is [Zmogus](#). Saugo varda, pavarde, pazymius, egzamino rezultata ir galutinius ivertinimus.

6.1.2 Constructor & Destructor Documentation

6.1.2.1 Studentas() [1/3]

```
Studentas::Studentas () [inline]
```

Numatytasis konstruktorius.

6.1.2.2 ~Studentas()

```
Studentas::~~Studentas () [inline], [override]
```

Destruktorius.

6.1.2.3 Studentas() [2/3]

```
Studentas::Studentas (  
    const Studentas & kitas) [inline]
```

Kopijavimo konstruktorius.

6.1.2.4 Studentas() [3/3]

```
Studentas::Studentas (  
    Studentas && kitas) [inline]
```

Perkelimo konstruktorius.

6.1.3 Member Function Documentation

6.1.3.1 getEgz()

```
int Studentas::getEgz () const [inline]
```

Grazina egzamino pazymi.

6.1.3.2 getGalutinisMed()

```
double Studentas::getGalutinisMed () const [inline]
```

Grazina galutini mediana.

6.1.3.3 getGalutinisVid()

```
double Studentas::getGalutinisVid () const [inline]
```

Grazina galutini vidurkiu.

6.1.3.4 getPavarde()

```
const std::string & Studentas::getPavarde () const [inline], [override], [virtual]
```

Grazina pavarde.

Implements [Zmogus](#).

6.1.3.5 getPaz()

```
const std::vector< int > & Studentas::getPaz () const [inline]
```

Grazina pazymiu vektoriu.

6.1.3.6 getRez()

```
double Studentas::getRez () const [inline]
```

Grazina rezultata.

6.1.3.7 getVardas()

```
const std::string & Studentas::getVardas () const [inline], [override], [virtual]
```

Grazina varda.

Implements [Zmogus](#).

6.1.3.8 isvalytiPaz()

```
void Studentas::isvalytiPaz () [inline]
```

Isvalo pazymius.

6.1.3.9 operator=() [1/2]

```
Studentas & Studentas::operator= (
    const Studentas & kitas) [inline]
```

Kopijavimo priskyrimo operatorius.

6.1.3.10 operator=() [2/2]

```
Studentas & Studentas::operator= (
    Studentas && kitas) [inline]
```

Perkelimo priskyrimo operatorius.

6.1.3.11 pridetiPaz()

```
void Studentas::pridetiPaz (
    int pazymys) [inline]
```

Prideda viena pazymi.

6.1.3.12 print()

```
void Studentas::print (
    std::ostream & os) const [inline], [override], [virtual]
```

Spausdina studento duomenis i srauta.

Implements [Zmogus](#).

6.1.3.13 read()

```
void Studentas::read (
    std::istream & in) [inline], [override], [virtual]
```

Nuskaito studento duomenis is srauto.

Implements [Zmogus](#).

6.1.3.14 setEgz()

```
void Studentas::setEgz (
    int naujasEgz) [inline]
```

Nustato egzamino pazymi.

6.1.3.15 setGalutinisMed()

```
void Studentas::setGalutinisMed (
    double naujasGalutinisMed) [inline]
```

Nustato galutini mediana.

6.1.3.16 setGalutinisVid()

```
void Studentas::setGalutinisVid (
    double naujasGalutinisVid) [inline]
```

Nustato galutini vidurkiu.

6.1.3.17 setPavarde()

```
void Studentas::setPavarde (
    const std::string & p) [inline], [override], [virtual]
```

Nustato pavarde.

Implements [Zmogus](#).

6.1.3.18 setPaz()

```
void Studentas::setPaz (
    const std::vector< int > & naujiPaz) [inline]
```

Nustato pazymius.

6.1.3.19 setRez()

```
void Studentas::setRez (
    double naujasRez) [inline]
```

Nustato rezultata.

6.1.3.20 setVardas()

```
void Studentas::setVardas (
    const std::string & v) [inline], [override], [virtual]
```

Nustato vardą.

Implements [Zmogus](#).

6.1.4 Member Data Documentation

6.1.4.1 egz

```
int Studentas::egz [private]
```

Egzamino pazymys.

6.1.4.2 galutinisMed

```
double Studentas::galutinisMed [private]
```

Galutinis pagal mediana.

6.1.4.3 galutinisVid

```
double Studentas::galutinisVid [private]
```

Galutinis pagal vidurkis.

6.1.4.4 paz

```
std::vector<int> Studentas::paz [private]
```

Namu darbu pazymiai.

6.1.4.5 rez

```
double Studentas::rez [private]
```

Rezultatas.

The documentation for this class was generated from the following file:

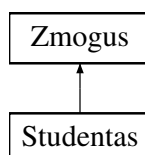
- [include/Studentas.h](#)

6.2 Zmogus Class Reference

Abstrakti bazine klase, aprasanti zmogu. Negalima tiesiogiai sukurti jos objektu.

```
#include <Zmogus.h>
```

Inheritance diagram for Zmogus:



Public Member Functions

- [Zmogus](#) ()=default
Numatytasis konstruktorius.
- [Zmogus](#) (const std::string &vardas, const std::string &pavarde)
Konstruktorius su parametrais.
- [Zmogus](#) ([Zmogus](#) &&kitas)
Perkelimo konstruktorius.
- virtual [~Zmogus](#) ()=default
Virtualus destruktorius.
- virtual const std::string & [getVardas](#) () const =0
Grazina vardą.
- virtual const std::string & [getPavarde](#) () const =0
Grazina pavardę.
- virtual void [setVardas](#) (const std::string &v)=0
Nustato vardą.
- virtual void [setPavarde](#) (const std::string &p)=0
Nustato pavardę.
- virtual void [print](#) (std::ostream &os) const =0
Spausdina objekto i srautą.
- virtual void [read](#) (std::istream &in)=0
Nuskaityti objekto is srauto.

Protected Attributes

- std::string [vardas](#)
Zmogaus vardas.
- std::string [pavarde](#)
Zmogaus pavardė.

Friends

- std::ostream & [operator<<](#) (std::ostream &os, const [Zmogus](#) &z)
Isvedimo operatorius.
- std::istream & [operator>>](#) (std::istream &in, [Zmogus](#) &z)
Ivedimo operatorius.

6.2.1 Detailed Description

Abstrakti bazinė klase, apimanti zmogų. Negalima tiesiogiai sukurti jos objektų.

6.2.2 Constructor & Destructor Documentation

6.2.2.1 Zmogus() [1/3]

```
Zmogus::Zmogus () [default]
```

Numatytasis konstruktorius.

6.2.2.2 Zmogus() [2/3]

```
Zmogus::Zmogus (
    const std::string & vardas,
    const std::string & pavarde) [inline]
```

Konstruktorius su parametrais.

6.2.2.3 Zmogus() [3/3]

```
Zmogus::Zmogus (
    Zmogus && kitas) [inline]
```

Perkelimo konstruktorius.

6.2.2.4 ~Zmogus()

```
virtual Zmogus::~Zmogus () [virtual], [default]
```

Virtualus destruktorius.

6.2.3 Member Function Documentation

6.2.3.1 getPavarde()

```
virtual const std::string & Zmogus::getPavarde () const [pure virtual]
```

Grazina pavarde.

Implemented in [Studentas](#).

6.2.3.2 getVardas()

```
virtual const std::string & Zmogus::getVardas () const [pure virtual]
```

Grazina varda.

Implemented in [Studentas](#).

6.2.3.3 print()

```
virtual void Zmogus::print (
    std::ostream & os) const [pure virtual]
```

Spausdina objekta i srauta.

Implemented in [Studentas](#).

6.2.3.4 read()

```
virtual void Zmogus::read (
    std::istream & in) [pure virtual]
```

Nuskaito objekta is rauto.

Implemented in [Studentas](#).

6.2.3.5 setPavarde()

```
virtual void Zmogus::setPavarde (
    const std::string & p) [pure virtual]
```

Nustato pavarde.

Implemented in [Studentas](#).

6.2.3.6 setVardas()

```
virtual void Zmogus::setVardas (
    const std::string & v) [pure virtual]
```

Nustato vardas.

Implemented in [Studentas](#).

6.2.4 Friends And Related Symbol Documentation

6.2.4.1 operator<<

```
std::ostream & operator<< (
    std::ostream & os,
    const Zmogus & z) [friend]
```

Isvedimo operatorius.

6.2.4.2 operator>>

```
std::istream & operator>> (
    std::istream & in,
    Zmogus & z) [friend]
```

Ivedimo operatorius.

6.2.5 Member Data Documentation

6.2.5.1 pavarde

```
std::string Zmogus::pavarde [protected]
```

Zmogaus pavarde.

6.2.5.2 vardas

```
std::string Zmogus::vardas [protected]
```

Zmogaus vardas.

The documentation for this class was generated from the following file:

- include/[Zmogus.h](#)

Chapter 7

File Documentation

7.1 include/funkcijos.h File Reference

```
#include "Zmogus.h"
#include "Studentas.h"
#include <vector>
#include <string>
#include <fstream>
#include <sstream>
#include <stdexcept>
#include <iostream>
#include <algorithm>
#include <type_traits>
#include <list>
#include <iomanip>
#include <deque>
#include <chrono>
```

Functions

- void [generuotiFaila](#) (const std::string &failoPavadinimas, int kiekStudentu, int ndKiekis)
- void [outputas](#) (const std::vector< [Studentas](#) > &grupe, char pasirinkimas)
- void [spausdinimas](#) (const std::vector< [Studentas](#) > &grupe, char pasirinkimas)
- void [inputas](#) (std::vector< [Studentas](#) > &grupe)
- double [mediana](#) (const std::vector< int > &paz)
- double [vidurkis](#) (const std::vector< int > &paz)
- void [tyrimas1](#) (const std::string &failoPavadinimas, int studentuKiekis, int ndKiekis)
- void [tyrimas2](#) (const std::string &failoPavadinimas, char budas)
- template<typename konteineris>
void [nuskaitymas](#) (konteineris &grupe, std::string failas)
- template<typename konteineris>
void [padalintiStudentus1](#) (const konteineris &grupe, konteineris &vargsiukai, konteineris &kietakai, char budas)
- template<typename konteineris>
void [padalintiStudentus2](#) (konteineris &grupe, konteineris &vargsiukai, char budas)
- template<typename konteineris>
void [padalintiStudentus3](#) (konteineris &grupe, konteineris &vargsiukai, char budas)

- `template<typename konteineris>`
`void rusiavimas (konteineris &grupe, char budas)`
- `template<typename konteineris>`
`void spausdintiFaila (const konteineris &grupe, const std::string &failoPavadinimas, char budas)`
- `template<typename konteineris>`
`void laikoSkaiciavimas (int strategija, int kriterijus, char budas, const std::string &konteinerioPavadinimas)`

7.1.1 Function Documentation

7.1.1.1 generuotiFaila()

```
void generuotiFaila (  
    const std::string & failoPavadinimas,  
    int kiekStudentu,  
    int ndKiekis)
```

7.1.1.2 inputas()

```
void inputas (  
    std::vector< Studentas > & grupe)
```

7.1.1.3 laikoSkaiciavimas()

```
template<typename konteineris>  
void laikoSkaiciavimas (  
    int strategija,  
    int kriterijus,  
    char budas,  
    const std::string & konteinerioPavadinimas)
```

7.1.1.4 mediana()

```
double mediana (  
    const std::vector< int > & paz)
```

7.1.1.5 nuskaitymas()

```
template<typename konteineris>  
void nuskaitymas (  
    konteineris & grupe,  
    std::string failas)
```

7.1.1.6 outputas()

```
void outputas (  
    const std::vector< Studentas > & grupe,  
    char pasirinkimas)
```

7.1.1.7 padalintiStudentus1()

```
template<typename konteineris>
void padalintiStudentus1 (
    const konteineris & grupe,
    konteineris & vargsiukai,
    konteineris & kietakai,
    char budas)
```

7.1.1.8 padalintiStudentus2()

```
template<typename konteineris>
void padalintiStudentus2 (
    konteineris & grupe,
    konteineris & vargsiukai,
    char budas)
```

7.1.1.9 padalintiStudentus3()

```
template<typename konteineris>
void padalintiStudentus3 (
    konteineris & grupe,
    konteineris & vargsiukai,
    char budas)
```

7.1.1.10 rusiavimas()

```
template<typename konteineris>
void rusiavimas (
    konteineris & grupe,
    char budas)
```

7.1.1.11 spausdinimas()

```
void spausdinimas (
    const std::vector< Studentas > & grupe,
    char pasirinkimas)
```

7.1.1.12 spausdintiIFaila()

```
template<typename konteineris>
void spausdintiIFaila (
    const konteineris & grupe,
    const std::string & failoPavadinimas,
    char budas)
```

7.1.1.13 tyrimas1()

```
void tyrimas1 (
    const std::string & failoPavadinimas,
    int studentuKiekis,
    int ndKiekis)
```

7.1.1.14 tyrimas2()

```
void tyrimas2 (
    const std::string & failoPavadinimas,
    char budas)
```

7.1.1.15 vidurkis()

```
double vidurkis (
    const std::vector< int > & paz)
```

7.2 funkcijos.h

[Go to the documentation of this file.](#)

```
00001 #ifndef FUNKCIJOS_H
00002 #define FUNKCIJOS_H
00003
00004 #include "Zmogus.h"
00005 #include "Studentas.h"
00006 #include <vector>
00007 #include <string>
00008 #include <fstream>
00009 #include <sstream>
00010 #include <stdexcept>
00011 #include <iostream>
00012 #include <algorithm>
00013 #include <type_traits>
00014 #include <list>
00015 #include <iomanip>
00016 #include <deque>
00017 #include <chrono>
00018
00019 void generuotiFaila(const std::string& failoPavadinimas, int kiekStudentu, int ndKiekis);
00020 void outputas(const std::vector<Studentas>& grupe, char pasirinkimas);
00021 void spausdinimas(const std::vector<Studentas>& grupe, char pasirinkimas);
00022 void inputas(std::vector<Studentas>& grupe);
00023 double mediana(const std::vector<int>& paz);
00024 double vidurkis(const std::vector<int>& paz);
00025 void tyrimas1(const std::string& failoPavadinimas, int studentuKiekis, int ndKiekis);
00026 void tyrimas2(const std::string& failoPavadinimas, char budas);
00027
00028 template <typename konteineris>
00029 void nuskaitymas(konteineris& grupe, std::string failas) {
00030     std::ifstream input(failas);
00031
00032     try {
00033         if (!input.is_open()) {
00034             throw std::runtime_error("Nepavyko atidaryti failo: " + failas);
00035         }
00036     } catch (std::exception& e) {
00037         std::cout << "Klaida: " << e.what() << std::endl;
00038         return;
00039     }
00040
00041     grupe.clear();
00042
00043     std::string eilute;
00044     std::getline(input, eilute);
00045
00046     while (std::getline(input, eilute)) {
```

```

00047         if (eilute.empty()) {
00048             continue;
00049         }
00050
00051         std::stringstream ss(eilute);
00052         Studentas s;
00053
00054         std::string vardas, pavarde;
00055         ss » vardas » pavarde;
00056
00057         s.setVardas(vardas);
00058         s.setPavarde(pavarde);
00059
00060         if (s.getVardas().empty() || s.getPavarde().empty()) {
00061             continue;
00062         }
00063
00064         std::vector<int> paz;
00065         int x;
00066
00067         while (ss » x) {
00068             paz.push_back(x);
00069         }
00070
00071         if (paz.empty()) {
00072             continue;
00073         }
00074
00075         s.setEgz(paz.back());
00076         paz.pop_back();
00077
00078         s.setPaz(paz);
00079         s.setGalutinisVid(0.4 * vidurkis(s.getPaz()) + 0.6 * s.getEgz());
00080         s.setGalutinisMed(0.4 * mediana(s.getPaz()) + 0.6 * s.getEgz());
00081
00082         grupe.push_back(s);
00083     }
00084 }
00085
00086 template <typename konteineris>
00087 void padalintiStudentus1(const konteineris& grupe, konteineris& vargsiukai, konteineris& kietakai,
00088     char budas) {
00089     vargsiukai.clear();
00089     kietakai.clear();
00090
00091     for (const auto& A : grupe) {
00092         double galutinis = (budas == 'm') ? A.getGalutinisMed() : A.getGalutinisVid();
00093
00094         if (galutinis < 5.0) {
00095             vargsiukai.push_back(A);
00096         } else {
00097             kietakai.push_back(A);
00098         }
00099     }
00100 }
00101
00102 template <typename konteineris>
00103 void padalintiStudentus2(konteineris& grupe, konteineris& vargsiukai, char budas) {
00104     vargsiukai.clear();
00105
00106     if constexpr (std::is_same_v<konteineris, std::list<Studentas>)> {
00107         grupe.sort([budas](const Studentas& A, const Studentas& B) {
00108             return (budas == 'm') ? A.getGalutinisMed() > B.getGalutinisMed() : A.getGalutinisVid() >
00109                 B.getGalutinisVid();
00110         });
00111     } else {
00112         std::sort(grupe.begin(), grupe.end(), [budas](const Studentas& A, const Studentas& B) {
00113             return (budas == 'm') ? A.getGalutinisMed() > B.getGalutinisMed() : A.getGalutinisVid() >
00114                 B.getGalutinisVid();
00115         });
00116     }
00117
00118     auto it = grupe.end();
00119
00120     while (it != grupe.begin()) {
00121         --it;
00122
00123         double galutinis = (budas == 'm') ? it->getGalutinisMed() : it->getGalutinisVid();
00124
00125         if (galutinis < 5.0) {
00126             vargsiukai.push_back(*it);
00127             it = grupe.erase(it);
00128         } else {
00129             break;
00130         }
00131     }
00132 }

```

```

00131
00132 template <typename konteineris>
00133 void padalintiStudentus3(konteineris& grupe, konteineris& vargsiukai, char budas) {
00134     vargsiukai.clear();
00135
00136     auto riba = std::partition(grupe.begin(), grupe.end(), [budas](const Studentas& A) {
00137         double galutinis = (budas == 'm') ? A.getGalutinisMed() : A.getGalutinisVid();
00138         return galutinis >= 5.0;
00139     });
00140
00141     vargsiukai.insert(vargsiukai.end(), riba, grupe.end());
00142     grupe.erase(riba, grupe.end());
00143 }
00144
00145 template <typename konteineris>
00146 void rusiavimas(konteineris& grupe, char budas) {
00147     int kriterijus;
00148     while (true) {
00149         std::cout << "Pasirinkite kriteriju pagal kuri norite rusiuoti:" << std::endl;
00150         std::cout << "1 - Vardas" << std::endl;
00151         std::cout << "2 - Pavarde" << std::endl;
00152         std::cout << "3 - Galutinis (vidurkis arba mediana)" << std::endl;
00153         std::cin >> kriterijus;
00154
00155         if (std::cin.fail()) {
00156             std::cin.clear();
00157             std::cin.ignore(10000, '\n');
00158             std::cout << "Klaida: iveskite skaiciu 1-3" << std::endl;
00159             continue;
00160         }
00161
00162         if (kriterijus == 1 || kriterijus == 2 || kriterijus == 3) {
00163             break;
00164         }
00165         std::cout << "Neteisinga ivestis. Bandykite dar karta!" << std::endl;
00166     }
00167
00168     if constexpr (std::is_same_v<konteineris, std::list<Studentas> >) {
00169         switch (kriterijus) {
00170             case 1:
00171                 grupe.sort([](const Studentas& A, const Studentas& B) {
00172                     return A.getVardas() < B.getVardas();
00173                 });
00174                 break;
00175             case 2:
00176                 grupe.sort([](const Studentas& A, const Studentas& B) {
00177                     return A.getPavarde() < B.getPavarde();
00178                 });
00179                 break;
00180             case 3:
00181                 if (budas == 'v') {
00182                     grupe.sort([](const Studentas& A, const Studentas& B) {
00183                         double galutinisA = 0.4 * vidurkis(A.getPaz()) + 0.6 * A.getEgz();
00184                         double galutinisB = 0.4 * vidurkis(B.getPaz()) + 0.6 * B.getEgz();
00185                         return galutinisA > galutinisB;
00186                     });
00187                 } else {
00188                     grupe.sort([](const Studentas& A, const Studentas& B) {
00189                         double galutinisA = 0.4 * mediana(A.getPaz()) + 0.6 * A.getEgz();
00190                         double galutinisB = 0.4 * mediana(B.getPaz()) + 0.6 * B.getEgz();
00191                         return galutinisA > galutinisB;
00192                     });
00193                 }
00194                 break;
00195         }
00196     } else {
00197         switch (kriterijus) {
00198             case 1:
00199             std::sort(grupe.begin(), grupe.end(), [](const Studentas& A, const Studentas& B) {
00200                 return A.getVardas() < B.getVardas();
00201             });
00202             break;
00203             case 2:
00204             std::sort(grupe.begin(), grupe.end(), [](const Studentas& A, const Studentas& B) {
00205                 return A.getPavarde() < B.getPavarde();
00206             });
00207             break;
00208             case 3:
00209             if (budas == 'v') {
00210                 std::sort(grupe.begin(), grupe.end(), [](const Studentas& A, const Studentas& B) {
00211                     double galutinisA = 0.4 * vidurkis(A.getPaz()) + 0.6 * A.getEgz();
00212                     double galutinisB = 0.4 * vidurkis(B.getPaz()) + 0.6 * B.getEgz();
00213                     return galutinisA > galutinisB;
00214                 });
00215             } else {
00216                 std::sort(grupe.begin(), grupe.end(), [](const Studentas& A, const Studentas& B) {
00217                     double galutinisA = 0.4 * mediana(A.getPaz()) + 0.6 * A.getEgz();

```



```

00218         double galutinisB = 0.4 * mediana(B.getPaz()) + 0.6 * B.getEgz();
00219         return galutinisA > galutinisB;
00220     });
00221 }
00222     break;
00223 }
00224 }
00225 }
00226
00227 template <typename konteineris>
00228 void spausdintiIFaila(const konteineris& grupe, const std::string& failoPavadinimas, char budas) {
00229     std::ofstream failas(failoPavadinimas);
00230
00231     failas << std::left << std::setw(15) << "Vardas" << std::setw(15) << "Pavarde" << std::setw(20) <<
"Galutinis" << std::endl;
00232
00233     failas << "-----" << std::endl;
00234
00235     for (const auto& A : grupe) {
00236
00237         double galutinis = (budas == 'm') ? A.getGalutinisMed() : A.getGalutinisVid();
00238
00239         failas << std::left << std::setw(15) << A.getVardas() << std::setw(15) << A.getPavarde() <<
std::setw(20) << std::fixed << std::setprecision(2) << galutinis << std::endl;
00240     }
00241
00242     failas.close();
00243 }
00244
00245 template <typename konteineris>
00246 void laikoSkaiciavimas(int strategija, int kriterijus, char budas, const std::string&
konteinerioPavadinimas) {
00247     std::vector<int> dydziai = {1000, 10000, 100000, 1000000, 10000000};
00248
00249     std::cout << "\n" << konteinerioPavadinimas << " konteineris:" << std::endl;
00250     std::cout << std::left << std::setw(12) << "Studentai" << std::setw(22) << "Nuskaitymo laikas" <<
std::setw(22) << "Rikiavimo laikas" << std::setw(22) << "Skirstymo laikas" << std::setw(22) << "Bendras
laikas" << std::endl;
00251
00252     for (int x : dydziai) {
00253         konteineris grupe;
00254         konteineris vargsiukai;
00255         konteineris kietakai;
00256
00257         auto start1 = std::chrono::high_resolution_clock::now();
00258         nuskaitymas(grupe, "studentai" + std::to_string(x) + ".txt");
00259         auto end1 = std::chrono::high_resolution_clock::now();
00260         std::chrono::duration<double> diff1 = end1 - start1;
00261
00262         auto start2 = std::chrono::high_resolution_clock::now();
00263
00264         if constexpr (std::is_same_v<konteineris, std::list<Studentas>>) {
00265             switch (kriterijus) {
00266                 case 1:
00267                     grupe.sort([](const Studentas& A, const Studentas& B) {
00268                         return A.getVardas() < B.getVardas();
00269                     });
00270                     break;
00271
00272                 case 2:
00273                     grupe.sort([](const Studentas& A, const Studentas& B) {
00274                         return A.getPavarde() < B.getPavarde();
00275                     });
00276                     break;
00277
00278                 case 3:
00279                     if (budas == 'v') {
00280                         grupe.sort([](const Studentas& A, const Studentas& B) {
00281                             return A.getGalutinisVid() > B.getGalutinisVid();
00282                         });
00283                     } else {
00284                         grupe.sort([](const Studentas& A, const Studentas& B) {
00285                             return A.getGalutinisMed() > B.getGalutinisMed();
00286                         });
00287                     }
00288                     break;
00289             }
00290         } else {
00291             switch (kriterijus) {
00292                 case 1:
00293                     std::sort(grupe.begin(), grupe.end(), [](const Studentas& A, const Studentas& B) {
00294                         return A.getVardas() < B.getVardas();
00295                     });
00296                     break;
00297
00298                 case 2:
00299                     std::sort(grupe.begin(), grupe.end(), [](const Studentas& A, const Studentas& B) {

```

```

00300         return A.getPavarde() < B.getPavarde();
00301     });
00302     break;
00303
00304     case 3:
00305     if (budas == 'v') {
00306         std::sort(grupe.begin(), grupe.end(), [](const Studentas& A, const Studentas& B) {
00307             return A.getGalutinisVid() > B.getGalutinisVid();
00308         });
00309     } else {
00310         std::sort(grupe.begin(), grupe.end(), [](const Studentas& A, const Studentas& B) {
00311             return A.getGalutinisMed() > B.getGalutinisMed();
00312         });
00313     }
00314     break;
00315 }
00316 }
00317
00318 auto end2 = std::chrono::high_resolution_clock::now();
00319 std::chrono::duration<double> diff2 = end2 - start2;
00320
00321 auto start3 = std::chrono::high_resolution_clock::now();
00322
00323 if (strategija == 1) {
00324     padalintiStudentus1(grupe, vargsiukai, kietakai, budas);
00325 } else if (strategija == 2) {
00326     padalintiStudentus2(grupe, vargsiukai, budas);
00327 } else {
00328     padalintiStudentus3(grupe, vargsiukai, budas);
00329 }
00330
00331 auto end3 = std::chrono::high_resolution_clock::now();
00332 std::chrono::duration<double> diff3 = end3 - start3;
00333
00334 double bendras = diff1.count() + diff2.count() + diff3.count();
00335
00336 std::cout << std::left << std::setw(12) << x << std::setw(22) << diff1.count() << std::setw(22) <<
00337     diff2.count() << std::setw(22) << diff3.count() << std::setw(22) << bendras << std::endl;
00338 }
00339 }
00340 #endif

```

7.3 include/Studentas.h File Reference

Studento klases aprasas.

```

#include "Zmogus.h"
#include <iostream>
#include <string>
#include <vector>
#include <sstream>

```

Classes

- class [Studentas](#)

Studento klase, paveldinti is [Zmogus](#). Saugo varda, pavarde, pazymius, egzamino rezultata ir galutinius ivertinimus.

7.3.1 Detailed Description

Studento klases aprasas.

Author

dziugasvas

Date

2026

7.4 Studentas.h

[Go to the documentation of this file.](#)

```

00001 #ifndef STUDENTAS_H
00002 #define STUDENTAS_H
00003
00004 #include "Zmogus.h"
00005 #include <iostream>
00006 #include <string>
00007 #include <vector>
00008 #include <sstream>
00009
00016
00022
00023 class Studentas : public Zmogus {
00024 private:
00025     std::vector<int> paz;
00026     int egz;
00027     double rez;
00028     double galutinisVid;
00029     double galutinisMed;
00030
00031 public:
00032     Studentas()
00033         : Zmogus(), egz(0), rez(0.0), galutinisVid(0.0), galutinisMed(0.0) {}
00034
00035     ~Studentas() override {
00036         vardas.clear();
00037         pavarde.clear();
00038         paz.clear();
00039         egz = 0;
00040         rez = 0.0;
00041         galutinisMed = 0.0;
00042         galutinisVid = 0.0;
00043     }
00044
00045     Studentas(const Studentas& kitas)
00046         : Zmogus(kitas.vardas, kitas.pavarde),
00047         paz(kitas.paz), egz(kitas.egz), rez(kitas.rez),
00048         galutinisVid(kitas.galutinisVid), galutinisMed(kitas.galutinisMed) {}
00049
00050     Studentas(Studentas&& kitas)
00051         : Zmogus(std::move(kitas)),
00052         paz(std::move(kitas.paz)),
00053         egz(std::move(kitas.egz)),
00054         rez(std::move(kitas.rez)),
00055         galutinisVid(std::move(kitas.galutinisVid)),
00056         galutinisMed(std::move(kitas.galutinisMed)) {
00057         kitas.egz = 0;
00058         kitas.rez = 0.0;
00059         kitas.galutinisVid = 0.0;
00060         kitas.galutinisMed = 0.0;
00061     }
00062
00063     Studentas& operator=(const Studentas& kitas) {
00064         if (this != &kitas) {
00065             vardas = kitas.vardas;
00066             pavarde = kitas.pavarde;
00067             paz = kitas.paz;
00068             egz = kitas.egz;
00069             rez = kitas.rez;
00070             galutinisVid = kitas.galutinisVid;
00071             galutinisMed = kitas.galutinisMed;
00072         }
00073         return *this;
00074     }
00075
00076     Studentas& operator=(Studentas&& kitas) {
00077         if (this != &kitas) {
00078             vardas = std::move(kitas.vardas);
00079             pavarde = std::move(kitas.pavarde);
00080             paz = std::move(kitas.paz);
00081             egz = std::move(kitas.egz);
00082             rez = std::move(kitas.rez);
00083             galutinisVid = std::move(kitas.galutinisVid);
00084             galutinisMed = std::move(kitas.galutinisMed);
00085             kitas.egz = 0;
00086             kitas.rez = 0.0;
00087             kitas.galutinisVid = 0.0;
00088             kitas.galutinisMed = 0.0;
00089         }
00090         return *this;
00091     }
00092
00093     const std::string& getVardas() const override { return vardas; }
00094
00095
00096
00097
00098
00099
00100

```

```

00101
00103     const std::string& getPavarde() const override { return pavarde; }
00104
00106     void setVardas(const std::string& v) override { vardas = v; }
00107
00109     void setPavarde(const std::string& p) override { pavarde = p; }
00110
00112     void print(std::ostream& os) const override {
00113         os << vardas << " " << pavarde << " ";
00114         for (int p : paz) os << p << " ";
00115         os << egz;
00116     }
00117
00119     void read(std::istream& in) override {
00120         std::string eilute;
00121         std::getline(in, eilute);
00122         std::stringstream ss(eilute);
00123         ss >> vardas >> pavarde;
00124         std::vector<int> visi;
00125         int skaicius;
00126         while (ss >> skaicius) visi.push_back(skaicius);
00127         if (!visi.empty()) {
00128             egz = visi.back();
00129             visi.pop_back();
00130         }
00131         paz = visi;
00132     }
00133
00135     const std::vector<int>& getPaz() const { return paz; }
00136
00138     int getEgz() const { return egz; }
00139
00141     double getRez() const { return rez; }
00142
00144     double getGalutinisVid() const { return galutinisVid; }
00145
00147     double getGalutinisMed() const { return galutinisMed; }
00148
00150     void setPaz(const std::vector<int>& naujiPaz) { paz = naujiPaz; }
00151
00153     void setEgz(int naujasEgz) { egz = naujasEgz; }
00154
00156     void setRez(double naujasRez) { rez = naujasRez; }
00157
00159     void setGalutinisVid(double naujasGalutinisVid) { galutinisVid = naujasGalutinisVid; }
00160
00162     void setGalutinisMed(double naujasGalutinisMed) { galutinisMed = naujasGalutinisMed; }
00163
00165     void pridetiPaz(int pazymys) { paz.push_back(pazymys); }
00166
00168     void isvalytiPaz() { paz.clear(); }
00169 };
00170
00171 #endif

```

7.5 include/Zmogus.h File Reference

Abstrakti bazine klase zmogui.

```

#include <string>
#include <iostream>

```

Classes

- class [Zmogus](#)

Abstrakti bazine klase, aprasanti zmogu. Negalima tiesiogiai sukurti jos objektu.

7.5.1 Detailed Description

Abstrakti bazine klase zmogui.

Author

dziugasvas

Date

2026

7.6 Zmogus.h

[Go to the documentation of this file.](#)

```
00001 #ifndef ZMOGUS_H
00002 #define ZMOGUS_H
00003
00004 #include <string>
00005 #include <iostream>
00006
00013
00019
00020 class Zmogus {
00021 protected:
00022     std::string vardas;
00023     std::string pavarde;
00024
00025 public:
00027     Zmogus() = default;
00028
00030     Zmogus(const std::string& vardas, const std::string& pavarde)
00031         : vardas(vardas), pavarde(pavarde) {}
00032
00034     Zmogus(Zmogus&& kitas)
00035         : vardas(std::move(kitas.vardas)), pavarde(std::move(kitas.pavarde)) {}
00036
00038     virtual ~Zmogus() = default;
00039
00041     virtual const std::string& getVardas() const = 0;
00042
00044     virtual const std::string& getPavarde() const = 0;
00045
00047     virtual void setVardas(const std::string& v) = 0;
00048
00050     virtual void setPavarde(const std::string& p) = 0;
00051
00053     virtual void print(std::ostream& os) const = 0;
00054
00056     virtual void read(std::istream& in) = 0;
00057
00059     friend std::ostream& operator<<(std::ostream& os, const Zmogus& z) {
00060         z.print(os);
00061         return os;
00062     }
00063
00065     friend std::istream& operator>>(std::istream& in, Zmogus& z) {
00066         z.read(in);
00067         return in;
00068     }
00069 };
00070
00071 #endif
```

7.7 src/funkcijos.cpp File Reference

```
#include "funkcijos.h"
#include <fstream>
#include <iostream>
#include <cstdlib>
#include <iomanip>
#include <stdexcept>
#include <algorithm>
#include <vector>
#include <chrono>
```

Functions

- void [generuotiFaila](#) (const std::string &failoPavadinimas, int studentuKiekis, int ndKiekis)
- void [outputas](#) (const vector< [Studentas](#) > &grupe, char pasirinkimas)
- void [spausdinimas](#) (const vector< [Studentas](#) > &grupe, char pasirinkimas)
- void [inputas](#) (vector< [Studentas](#) > &grupe)
- double [mediana](#) (const vector< int > &paz)
- double [vidurkis](#) (const vector< int > &paz)
- void [tyrimas1](#) (const string &failoPavadinimas, int studentuKiekis, int ndKiekis)
- void [tyrimas2](#) (const string &failoPavadinimas, char budas)

7.7.1 Function Documentation

7.7.1.1 [generuotiFaila\(\)](#)

```
void generuotiFaila (
    const std::string & failoPavadinimas,
    int studentuKiekis,
    int ndKiekis)
```

7.7.1.2 [inputas\(\)](#)

```
void inputas (
    vector< Studentas > & grupe)
```

7.7.1.3 [mediana\(\)](#)

```
double mediana (
    const vector< int > & paz)
```

7.7.1.4 [outputas\(\)](#)

```
void outputas (
    const vector< Studentas > & grupe,
    char pasirinkimas)
```

7.7.1.5 spausdinimas()

```
void spausdinimas (
    const vector< Studentas > & grupe,
    char pasirinkimas)
```

7.7.1.6 tyrimas1()

```
void tyrimas1 (
    const string & failoPavadinimas,
    int studentuKiekis,
    int ndKiekis)
```

7.7.1.7 tyrimas2()

```
void tyrimas2 (
    const string & failoPavadinimas,
    char budas)
```

7.7.1.8 vidurkis()

```
double vidurkis (
    const vector< int > & paz)
```

7.8 src/main.cpp File Reference

```
#include <iostream>
#include <iomanip>
#include <cstdlib>
#include <ctime>
#include <chrono>
#include <list>
#include <deque>
#include <cctype>
#include "funkcijos.h"
```

Functions

- int [main](#) ()

Variables

- const vector< string > [vardai](#) = {"Dovydas", "Matas", "Simonas", "Rokas", "Kajus", "Dziugas", "Virgilijus", "Vitalijus", "Alan", "Aleksas", "Jonas", "Domantas", "Arvydas", "Mantyvdas", "Gvidas"}
- const vector< string > [pavardes](#) = {"Kazlauskas", "Buzelis", "Sabonis", "Tubelis", "Gudelis", "Macijauskas", "Aleksa", "Vanagas", "Butkevicius", "Ulanovas", "Sirvydis", "Jasikevicius", "Jakucionis", "Kleiza", "Jonauskas"}

7.8.1 Function Documentation

7.8.1.1 main()

```
int main ()
```

7.8.2 Variable Documentation

7.8.2.1 pavardes

```
const vector<string> pavardes = {"Kazlauskas", "Buzelis", "Sabonis", "Tubelis", "Gudelis",  
"Macijauskas", "Aleksa", "Vanagas", "Butkevicius", "Ulanovas", "Sirvydis", "Jasikevicius",  
"Jakucionis", "Kleiza", "Jonas"};
```

7.8.2.2 vardai

```
const vector<string> vardai = {"Dovydas", "Matas", "Simonas", "Rokas", "Kajus", "Dziugas",  
"Virgilijus", "Vitalijus", "Alan", "Aleksas", "Jonas", "Domantas", "Arvydas", "Mantyvdas",  
"Gvidas"};
```


Index

- ~Studentas
 - Studentas, [13](#)
- ~Zmogus
 - Zmogus, [19](#)
- egz
 - Studentas, [17](#)
- funkcijos.cpp
 - generuotiFaila, [32](#)
 - inputas, [32](#)
 - mediana, [32](#)
 - outputas, [32](#)
 - spausdinimas, [32](#)
 - tyrimas1, [33](#)
 - tyrimas2, [33](#)
 - vidurkis, [33](#)
- funkcijos.h
 - generuotiFaila, [22](#)
 - inputas, [22](#)
 - laikoSkaiciavimas, [22](#)
 - mediana, [22](#)
 - nuskaitymas, [22](#)
 - outputas, [22](#)
 - padalintiStudentus1, [22](#)
 - padalintiStudentus2, [23](#)
 - padalintiStudentus3, [23](#)
 - rusiavimas, [23](#)
 - spausdinimas, [23](#)
 - spausdintiFaila, [23](#)
 - tyrimas1, [23](#)
 - tyrimas2, [24](#)
 - vidurkis, [24](#)
- galutinisMed
 - Studentas, [17](#)
- galutinisVid
 - Studentas, [17](#)
- generuotiFaila
 - funkcijos.cpp, [32](#)
 - funkcijos.h, [22](#)
- getEgz
 - Studentas, [13](#)
- getGalutinisMed
 - Studentas, [13](#)
- getGalutinisVid
 - Studentas, [14](#)
- getPavarde
 - Studentas, [14](#)
 - Zmogus, [19](#)
- getPaz
 - Studentas, [14](#)
- getRez
 - Studentas, [14](#)
- getVardas
 - Studentas, [14](#)
 - Zmogus, [19](#)
- include Directory Reference, [9](#)
- include/funkcijos.h, [21](#), [24](#)
- include/Studentas.h, [28](#), [29](#)
- include/Zmogus.h, [30](#), [31](#)
- inputas
 - funkcijos.cpp, [32](#)
 - funkcijos.h, [22](#)
- isvalytiPaz
 - Studentas, [14](#)
- laikoSkaiciavimas
 - funkcijos.h, [22](#)
- main
 - main.cpp, [34](#)
- main.cpp
 - main, [34](#)
 - pavardes, [34](#)
 - vardai, [34](#)
- mediana
 - funkcijos.cpp, [32](#)
 - funkcijos.h, [22](#)
- nuskaitymas
 - funkcijos.h, [22](#)
- operator<<
 - Zmogus, [20](#)
- operator>>
 - Zmogus, [20](#)
- operator=
 - Studentas, [14](#), [15](#)
- outputas
 - funkcijos.cpp, [32](#)
 - funkcijos.h, [22](#)
- padalintiStudentus1
 - funkcijos.h, [22](#)
- padalintiStudentus2
 - funkcijos.h, [23](#)
- padalintiStudentus3
 - funkcijos.h, [23](#)
- pavarde

- Zmogus, 20
- pavardes
 - main.cpp, 34
- paz
 - Studentas, 17
- pridetiPaz
 - Studentas, 15
- print
 - Studentas, 15
 - Zmogus, 19
- read
 - Studentas, 15
 - Zmogus, 19
- rez
 - Studentas, 17
- rusiavimas
 - funkcijos.h, 23
- setEgz
 - Studentas, 15
- setGalutinisMed
 - Studentas, 15
- setGalutinisVid
 - Studentas, 16
- setPavarde
 - Studentas, 16
 - Zmogus, 20
- setPaz
 - Studentas, 16
- setRez
 - Studentas, 16
- setVardas
 - Studentas, 16
 - Zmogus, 20
- spausdinimas
 - funkcijos.cpp, 32
 - funkcijos.h, 23
- spausdintiFaila
 - funkcijos.h, 23
- src Directory Reference, 9
- src/funkcijos.cpp, 32
- src/main.cpp, 33
- Studentas, 11
 - ~Studentas, 13
 - egz, 17
 - galutinisMed, 17
 - galutinisVid, 17
 - getEgz, 13
 - getGalutinisMed, 13
 - getGalutinisVid, 14
 - getPavarde, 14
 - getPaz, 14
 - getRez, 14
 - getVardas, 14
 - isvalytiPaz, 14
 - operator=, 14, 15
 - paz, 17
 - pridetiPaz, 15
 - print, 15
 - read, 15
 - rez, 17
 - setEgz, 15
 - setGalutinisMed, 15
 - setGalutinisVid, 16
 - setPavarde, 16
 - setPaz, 16
 - setRez, 16
 - setVardas, 16
 - Studentas, 13
- tyrimas1
 - funkcijos.cpp, 33
 - funkcijos.h, 23
- tyrimas2
 - funkcijos.cpp, 33
 - funkcijos.h, 24
- vardai
 - main.cpp, 34
- vardas
 - Zmogus, 20
- vidurkis
 - funkcijos.cpp, 33
 - funkcijos.h, 24
- Zmogus, 17
 - ~Zmogus, 19
 - getPavarde, 19
 - getVardas, 19
 - operator<<, 20
 - operator>>, 20
 - pavarde, 20
 - print, 19
 - read, 19
 - setPavarde, 20
 - setVardas, 20
 - vardas, 20
 - Zmogus, 18, 19